

SkeletonSR (BasicSR): Pseudocode

Detailed version ($\approx 80\%$ of the source)

Overview. SkeletonSR is a minimal PySR-style symbolic-regression engine. The fixed *skeleton* (this document) handles tree representation, budgets, random tree construction, constraints, constant optimisation and the outer evolutionary loop. Everything that decides *which* expressions win is delegated to a pluggable *policy* with the slots shown in **BLUE SMALL CAPS**: `INIT_STATE`, `LOSS_FUNCTION`, `SELECTION`, `MUTATION`, `CROSSOVER`, `ACCEPTANCE`, `SURVIVAL`, `UPDATE_POPULATION`, `UPDATE_STATE!`, plus the optional `CYCLES_PER_POPULATION` and `SHOULD_CROSSOVER`.

Data structures

- 1: **Node** ::= `ConstNode(value)` | `VarNode(feature)` | `OpNode(op, left, right |  $\emptyset$ )`
- 2: **Individual** = `(tree, loss, cost, complexity, birth, ref, parent_ref)`
- 3: **EngineConfig** = `(population_size, populations, niterations, ncycles_per_iteration, maxsize, maxdepth, max_evals, timeout, binary_ops, unary_ops, constants, constraints, nested_constraints, seed)`
- 4: **EvolutionEngine** = `(X, y, cfg, rng, counters: birth/ref/eval_count, eval_budget, start_time)`
- 5: **EngineState** = `(engine, populations, policy_state, current_iteration / population / inner_cycle)`
- 6: `tree_size(n)`, `tree_height(n)`: recursive node count / depth.
- 7: `evaluate_tree(n, X)`: recursive broadcast evaluation with NaN-safe ops (`safe_log`, `safe_sqrt`, `safe_pow`, ...).

Budget

- 1: **function** `HASBUDGET(engine)`
- 2: **return** `(eval_budget =  $\emptyset \vee$  eval_count < eval_budget)  $\wedge$  (timeout =  $\emptyset \vee$  elapsed < timeout)`

Scoring an expression

- 1: **function** `MAKEINDIVIDUAL(tree, state, config, parent_ref)`
- 2: **if** $\neg$ `HASBUDGET(engine)` **then return** `$\emptyset$`
- 3: `eval_count` $\leftarrow$ `eval_count` + 1
- 4: `(loss, cost)` $\leftarrow$ `LOSS_FUNCTION(tree, tree_size(tree), state, config)` $\triangleright \emptyset$ if rejected
- 5: **return** `Individual(tree, loss, cost, tree_size(tree), next_birth, next_ref, parent_ref)`

Random tree construction

- 1: **function** `RANDOMTERMINAL`

```

2:   with prob.  $\frac{1}{2}$ : VarNode(random feature); else ConstNode(random from cfg.constants, or  $\mathcal{N}(0, 1)$ )
3: function APPENDRANDOMOP(tree)
4:   pick a random leaf; sample arity  $\in \{1, 2\}$  weighted by #unary : #binary ops; sample op of
   that arity
5:   replace the leaf with OpNode(op, RandomTerminal, [RandomTerminal])
6: Also available to policies: PrependRandomOp (wrap root), InsertRandomOp (wrap random node),
   RandomTreeFixedSize( $k$ ) (append until  $k$  nodes), RandomTree(max_depth, full) (grow/full re-
   cursion).
7: function VALIDTREE(tree)
8:   return tree_size  $\leq$  maxsize  $\wedge$  tree_height  $\leq$  maxdepth  $\wedge$  CHECKCONSTRAINTS(tree)
9: function CHECKCONSTRAINTS(tree)
10:  for each OpNode  $n$  in tree do
11:    if constraints[ $n$ .op] set: child subtree sizes must not exceed the per-argument limits
12:    if nested_constraints[ $n$ .op] set: nesting depth of each listed child op under  $n$  must not
    exceed its limit
13:  return true if all pass

```

Population initialisation

```

1: function INITIALIZEPOPULATION(state, config)
2:   pop  $\leftarrow []$ 
3:   while |pop| < population_size  $\wedge$  HASBUDGET do
4:     tree  $\leftarrow$  ConstNode(0); apply APPENDRANDOMOP three times
5:     if  $\neg$ VALIDTREE(tree) then tree  $\leftarrow$  RANDOMTREE(min(3, maxdepth), full=false)
6:      $m \leftarrow$  MAKEINDIVIDUAL(tree); push if  $m \neq \emptyset$ 
7:     if pop empty then push the constant  $\bar{y}$  (with loss  $\infty$  if it cannot be scored)
8:   return pop

```

Constant optimisation and simplification

```

1: function OPTIMIZECONSTANTS(member)
2:   consts  $\leftarrow$  constant nodes of member.tree; return member if none or eval budget  $\leq 1$ 
3:   starts  $\leftarrow$  {current values}  $\cup$  {nrestarts copies perturbed by  $\times(1 + 0.5\mathcal{N}(0, 1))$ }
4:   algorithm  $\leftarrow$  Newton if single constant and enabled, else Nelder–Mead; cap f_calls by remaining
   budget
5:   for each  $x_0 \in$  starts while HASBUDGET do
6:     minimise vals  $\mapsto$  LOSS_FUNCTION(tree with constants = vals) (each call costs one eval)
7:     keep best (tree, loss) if improved; on optimiser error, warn and continue
8:   if improved then re-score best tree once more; return new Individual with parent_ref =
   member.ref
9:   return member
10: function SIMPLIFYTREE(tree)
11:   recursively fold OpNode(op, Const, Const) into a Const (clamped to  $\pm 10^6$ ) when finite
12: function OPTIMIZEANDSIMPLIFY!(population)
13:   for each member in population do
14:     if should_simplify then  $s \leftarrow$  SIMPLIFYTREE; adopt if VALIDTREE( $s$ )

```

```

15:     if should_optimize_constants  $\wedge$  rand < optimize_probability then member  $\leftarrow$  OPTIMIZE-
    CONSTANTS(member)

```

Main loop

```

1: function FITSKELETONSR( $X, y$ , config)
2:   state  $\leftarrow$  EvolutionEngine( $X, y$ , cfg) + INIT_STATE(config)
3:   state.populations  $\leftarrow$  [INITIALIZEPOPULATION for  $1..n_{\text{pops}}$ ]
4:   UPDATE_STATE!(populations, state, config)
5:   for iteration = 1 ... niterations while HASBUDGET do
6:     for each population index  $p$  while HASBUDGET do
7:       for inner = 1 ... ncycles_per_iteration while HASBUDGET do
8:         UPDATE_STATE!(populations, state, config)
9:         EVOLVECYCLE!(state,  $p$ )
10:        record trace snapshot if a milestone (in evals or iterations) was crossed
11:        OPTIMIZEANDSIMPLIFY!(populations[ $p$ ]) (if budget remains)
12:        UPDATE_STATE!(populations, state, config)
13:        populations  $\leftarrow$  UPDATE_POPULATION(policy_state, populations, state, config)  $\triangleright$  e.g.
        migration, archive
14:   return archived (or best) expressions sorted by complexity, plus n_evals and the trace

1: function EVOLVECYCLE!(state,  $p$ )
2:   pop  $\leftarrow$  populations[ $p$ ]
3:   for 1 ... CYCLES_PER_POPULATION(pop) do  $\triangleright$  default:  $\lceil |pop|/tournament_n \rceil$  rounds
4:     if  $\neg$ HASBUDGET then break
5:     if SHOULD_CROSSOVER(pop) then  $\triangleright$  default: rand  $\leq$  crossover_probability
6:        $a \leftarrow$  SELECTION(pop);  $b \leftarrow$  SELECTION(pop); if  $a \equiv b$ , take the next member as  $b$ 
7:       trees  $\leftarrow$  CROSSOVER( $a, b$ )  $\triangleright$  may return 0, 1 or 2 trees
8:       candidates  $\leftarrow$  [child for (tree, parent) in zip(trees, ( $a, b$ )) if child  $\leftarrow$  MAKEINDIVIDUAL
and ACCEPTANCE(parent, child)]
9:       if candidates non-empty then pop  $\leftarrow$  SURVIVAL(pop, candidates)
10:    else
11:      parent  $\leftarrow$  SELECTION(pop)
12:      result  $\leftarrow$  MUTATION(parent)
13:      if result =  $\emptyset$  then continue
14:      else if result is an Individual then pop  $\leftarrow$  SURVIVAL(pop, [result])  $\triangleright$  already scored by
the policy
15:    else
16:      child  $\leftarrow$  MAKEINDIVIDUAL(result, parent_ref = parent.ref); return if  $\emptyset$  (budget
exhausted)
17:    if ACCEPTANCE(parent, child) then pop  $\leftarrow$  SURVIVAL(pop, [child])
18:    populations[ $p$ ]  $\leftarrow$  pop

```